CausaliT: Towards an Intervention Invariant Transformer

Francesco Scipione
fscipion@ethz.ch

Abstract

Transformers are powerful sequence-to-sequence models widely adopted beyond their original NLP tasks. An interesting application is the use of transformers as data-driven surrogates for optimization, where the ability to reliably predict under interventions on input variables is crucial. Transformers can be reliable surrogate models for optimization only if they capture interventional invariances; providing the true *directed acyclic graph* (DAG) induced by the data-generating process guarantees this, but in practical applications, the true DAG is often unknown. Therefore, learning the DAG from data is fundamental, and this work highlights the major challenges transformers face, proposing, at the same time, an architecture tailored for causal discovery[1].

1 Motivation

Consider a specific use of transformers [1], where they are employed as data-driven surrogates for black-box systems in optimization problems. A practical example of this is manufacturing, where the operators set recipe values $\mathbf{S} = [S_1, \dots, S_M]$ and the machine produces log files with state variables $\mathbf{X} = [X_1, \dots, X_D]$, leading to the final measurable target $\mathbf{Y} = [Y_1, \dots, Y_S]$, which can be any quantifiable feature of the product. We distinguish between $\mathbf{S}$ and $\mathbf{X}$ as the former are controllable by the operator, while the latter are produced by the machine and represent its state in a given moment in time. Given a surrogate $f(\mathbf{S}) = \hat{\mathbf{Y}}$, a typical optimization problem (optimizing the input recipe $\mathbf{S}$, for example) is to minimize a cost function $\mathcal{L}(\hat{\mathbf{Y}}, \mathbf{Y})$

$$\mathbf{S}^\star = \arg_{\mathbf{S}} \min \mathcal{L}(f(\mathbf{S}), \mathbf{Y}) \tag{1}$$

and if f is differentiable, then gradient-based optimization is viable. Independently of the optimization strategy, f should be able to model interventions, i.e. $\mathbb{E}[\hat{\mathbf{Y}}|\mathbf{S} = s'] = \mathbb{E}^{\mathrm{do}(\mathbf{S}:=s')}[\mathbf{Y}]$ to correctly guide the algorithm towards realistic solutions $\mathbf{S}^\star$. In this work, we focus on a transformer architecture for the surrogate f, which can encode known causal knowledge of the data-generating process [2].

We assume the following structural hierarchy $\mathbf{S} \rightharpoonup \mathbf{X} \rightharpoonup \mathbf{Y}$, which means that $\mathrm{PA}(\mathbf{X}) \in \mathbf{S}$ <u>and</u> $\mathrm{PA}(\mathbf{Y}) \in \mathbf{X}$ <u>and</u> $\mathrm{PA}(\mathbf{Y}) \notin \mathbf{S}$. In specific applications, this is a reasonable assumption. As an example, consider modeling a manufacturing machine. Let $\mathbf{S}$ be the set of *settings* the operator can decide from; they are set-points, from a control-theoretic prospective and can be chosen arbitrarily from a set of available discrete or continuous values. Now, let $\mathbf{X}$ be the machine parameters that the operator has no control over. They could be default safety options, sensor measurements of actual physical quantities, and downstream log values, in general, describing the machine *state*. Such parameters are caused by the settings, not vice-versa, and therefore can be assumed to follow the first structural hierarchy $\mathbf{S} \rightharpoonup \mathbf{X}$. Given these premises, we can further assume that the machine's state is sufficiently well described by its state variables $\mathbf{X}$, which mix information from the settings with noise coming from different sources: sensor drift, environmental changes, and maintenance distance, directly affecting the output, i.e., $\mathbf{X} \rightharpoonup \mathbf{Y}$.

[1] Project repo https://github.com/scipi1/causaliT

2 Transformer as a Factorization Engine

The following *structured causal model* (SCM) is used to generate a synthetic dataset of 50k covariates.

$$
\begin{aligned}
S_1 &= N_{S_1} \\
S_2 &= N_{S_2} \\
S_3 &= N_{S_3} \\
X_2 &= g_{X_2}(S_2, S_3) + N_{X_2} \\
X_1 &= g_{X_1}(S_1, X_2) + N_{X_1} \\
Y_1 &= g_{Y_1}(X_1) + N_{Y_1} \\
Y_2 &= g_{Y_1}(X_2) + N_{Y_2}
\end{aligned}
$$

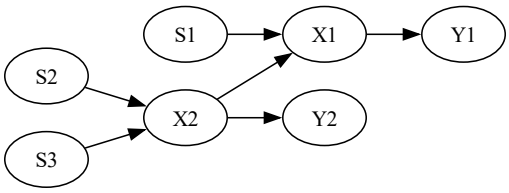


Figure 1: Induced DAG

The noise variables $N_{S_1}, N_{S_2}, N_{S_3} \sim \mathcal{U}(-1, 1)$ are used to uniformly sample the source values. The remaining independent noise variables $\sim \mathcal{N}(0, 0.1)$. All g functions are linear with unitary slope and null bias.

The adjacency matrix of the direct acyclic graph (DAG) can be effectively encoded into the attention blocks of a transformer, as shown in Figure 2. We use a two-decoder transformer, a non-autoregressive model similar to the *Informer* [3], that encodes the causal hierarchy $\mathbf{S} \rightharpoonup \mathbf{X} \rightharpoonup \mathbf{Y}$.

NB: With a slight abuse of notation, we call both the input sequence and the embedded sequence by the same name, as it gets updated as it passes through the transformer blocks.

Embeddings Consider the input sample $\mathbf{X} = [X_1, X_2] \in \mathbb{R}^L$, with $L = 2$. The embedding blocks compute rich representations (d-dimensional tokens) of each variable, fusing information of the position j along the sequence length (or variable ID in our case) and the realization, x_j, i.e. $X_j := \text{emb}(j) + \text{emb}(x_j) \in \mathbb{R}^{L \times d}$.

Inverted Stage Decoder $\mathbf{X}$ updates its representation from $\mathbf{S}$ through the cross-attention block. At this point, $\mathbf{X}$ carries information Of their structural parents, but the true edges $S_j \to X_k$ must be learned. Next, the "enriched" $\mathbf{X}$ updates its representation from all other $\mathbf{X}$s in the sequence *given* the information they inherit from the eventual $\mathbf{S}$ parents. This caveat motivates inverting cross-attention and self-attention in our decoder, compared to the standard self-to-cross attention configuration. For the next stage, everything repeats: the $\mathbf{X}$ are now the sources and $\mathbf{Y}$ are the sinks. At every stage, the loss component is computed as a cost function of the predictions and the targets.

Attention as token mixer [2] Consider the embedded input sequence $\mathbf{X} \in \mathbb{R}^{L \times d}$, where L is the sequence length and d is the hidden size. Attention generates queries $\mathbf{Q}$, keys $\mathbf{K}$ and values $\mathbf{V}$ through linear maps $W_Q, W_K \in \mathbb{R}^{d \times d_{QK}}$, and $W_V \in \mathbb{R}^{d \times d}$, with $d_{QK} < d$. At this point, tokens mix with each other, i.e., they are processed with all other tokens along the sequence length L in the following way.

1. A similarity metric $\alpha \in \mathbb{R}^{L \times L}$, commonly referred to as **the attention score**, measures how close queries and keys are in d-space through a kernel $k(\mathbf{Q}, \mathbf{K})$ [4].
2. Attention scores are used to pass information from values $\mathbf{V}$ to queries $\mathbf{Q}$ using a weighted sum $A(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \alpha \mathbf{V} = \alpha_1 \mathbf{V}_{1:} + \cdots + \alpha_L \mathbf{V}_{L:}$. The notation ":" selects all columns of the hidden dimension d. The result is a mix of the value representations, pondered by scores computed on a similarity metric. Higher similarity nudges more information flow between the respective tokens.
3. Finally, the residual connection updates the sequence $\mathbf{X} := \mathbf{X} + A(\mathbf{Q}, \mathbf{K}, \mathbf{V})$

The most common kernel is the scaled dot product and SoftMax [1], but many variations have been proposed, all working in a similar way to calculate an *alignment* [5]. If attention is the only mechanism for mixing tokens along L, enforcing causality in these blocks ensures it for the full model.

[2] The example is given for self attention, where the queries, keys and values have the same sequence length. For cross-attention, we will have two lengths L_Q and L_K and $\alpha \in \mathbb{R}^{L_Q \times L_K}$

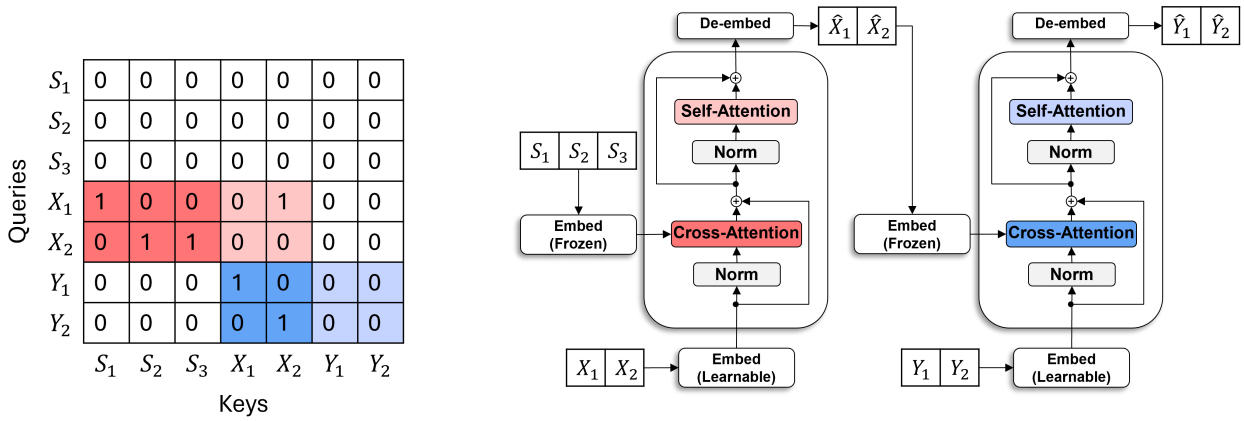


Figure 2: The LHS panel shows the adjacency matrix of the DAG in 1. Entries at ij are one if the edge $j \to i$ exists, zero otherwise. The RHS panel contains the block diagram of the two-decoder transformer. Colors match DAG slices in the adjacency matrix with the respective attention blocks in the transformer.

3 When Causality is Given

Consider an attention block A, with queries and keys of length L_Q and L_K, respectively. The true DAG of the block is defined as a mask $M \in \{0,1\}^{L_Q \times L_K}$, where

$$M_{ij} \begin{cases} 0 \ (-\infty \text{ for SoftMax}) & \nexists \text{ edge } j \to i \\ 1 \ (0 \text{ for SoftMax}) & \exists \text{ edge } j \to i \end{cases} \tag{2}$$

We can generally apply the DAG to the attention scores α using the Hadamard product $\alpha \odot M$. An important exception is when the scores are computed with the SoftMax. In that case, the mask must be applied before activation by summation: $\text{SoftMax}(QK^\top/\sqrt{d_{QK}} + M)$. There are two reasons for this exception.

1. the SoftMax is zero only at $-\infty$.
2. SoftMax normalizes the queries to a unitary sum along the keys, providing a probabilistic interpretation of the average sum. If we apply the mask as an element-wise product after the SoftMax, interventions on non-ancestors of X would still influence X through such a normalization.

The application of the true DAG to a transformer attention block has already been proposed by [2] as a simple method to enforce causal behavior.

Measure intervention invariance

Intervention invariance is measured by comparing $\mathbb{E}[Y_2|S_1 := s']$ and the expected $\mathbb{E}^{\text{do}(S_1:=s')}[Y_2]$ from the SCM. As in $S_1 \notin \text{AN}(Y_2)$, we expect $\mathbb{E}^{\text{do}(S_1:=s)}[Y_2] = \mathbb{E}[Y_2]$ in causal predictors. We trained a transformer with and without the true DAG. Then, we performed predictions on the test split by setting $S_1 := s'$, with $s' \in \{1, 0, -1\}$. The sample deviations along feature j are calculated as $\Delta Y_{j,i} = \hat{Y}_{j,i} - \hat{Y}_{j,i}^{\text{do}(S_1:=s)}$ and reported in Figure 3.

Results

When provided with the true DAG, the transformer is guaranteed to be intervention-invariant. It learns a map $f : \mathbf{S} \mapsto \mathbf{Y}$ such that

$$\mathbb{E}[Y_j|S_i := s] = \mathbb{E}[Y_j] \text{ if } S_i \notin \text{AN}(Y_j)$$

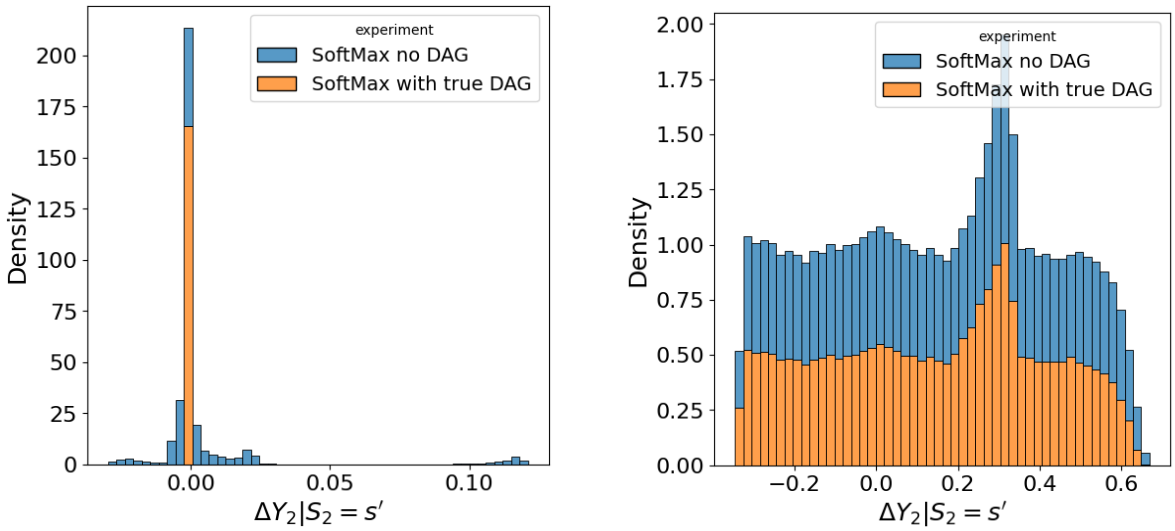


Figure 3: Measured distribution shift under the interventions $S_1, S_2 = s'$, with $s' \in \{0, -5, 5\}$. We used a two-decoder architecture, scaled dot-product attention, and SoftMax activation. The dataset is linear Gaussian. [Experiments 54604038, 54603903. Dataset scm6.]

4 Residuals Analysis

The intervention experiment in section 3 highlighted the need to provide the true DAG to the model to preserve intervention-invariance. If the true DAG is known with absolute certainty, ensuring intervention invariance in the transformer is trivial. In reality, the true DAG is often elusive, and exhaustive knowledge of it is unrealistic. In this section, we examine the consequences of providing the wrong DAG to the transformer by focusing on a crucial metric in causality: the residuals-parents independence.

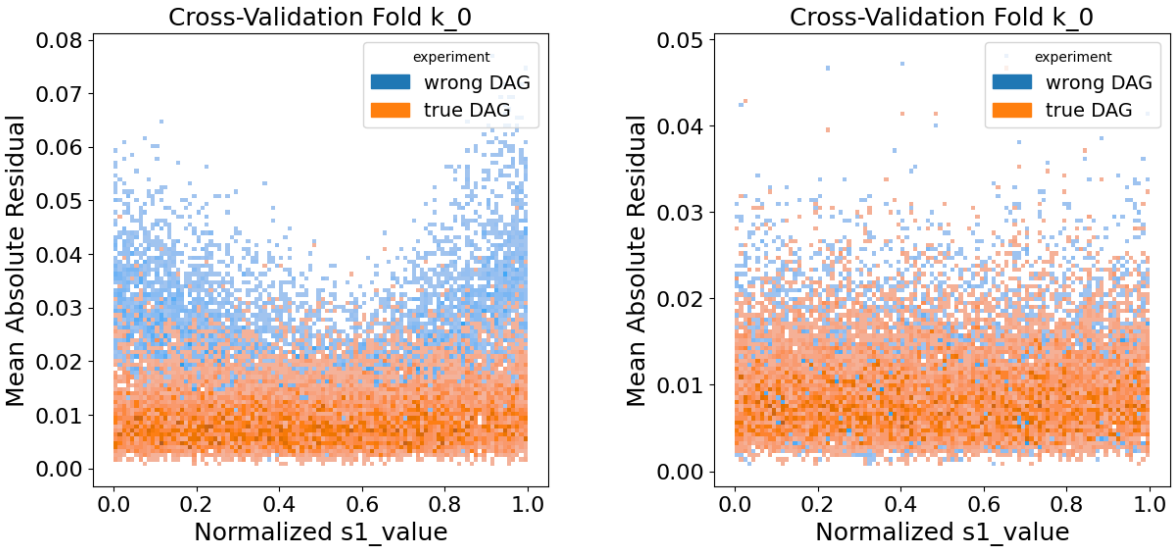
A known result, often used in causal discovery, is the residuals-parents independence [6]. Given a transformer $\hat{\mathbf{Y}} = f(\mathbf{S}) = \mathbb{E}[\mathbf{Y}|\mathbf{S}]$, residuals $\epsilon = \mathbf{Y} - \hat{\mathbf{Y}}$, we expect $\epsilon \perp \mathbf{S}$ if the regressor encodes the true parent-child relations. When the real DAG is provided to the model, causality is enforced and high independence between the residuals and $\mathbf{S}$ is expected. In this section, we investigate the case in which the *wrong* DAG is used instead.

Residuals for true and wrong DAG

We derived a wrong DAG by altering edges on the true DAG and trained the same model with this only variation. Then, the independence between the residuals and S is measured by the Hilbert-Schmidt Independence Criterion (HSIC). The results are reported in Table 1, and a scatter plot is shown in Figure 4. We tested the two-decoder transformer with SoftMax attention in two parameterization regimes: over-parametrized and under-parametrized, to see the effect of model capacity on the HSIC.

Results

- The true DAG leads to higher residuals-parents independence than the wrong one, in both the over-parametrized and under-parametrized regimes, resulting in a strictly lower HSIC value.
- The HSIC is not uniquely determined by deviations from the true DAG. Higher model capacity reduces the HSIC, and poor fit dramatically increases the HSIC, even if the true DAG is given (see Figure 9).



(a) Under-parametrized

(b) Over-parametrized

Figure 4: The figure shows the scatter plot of residuals and parent values (only S_1 here) for a model provided with the true and the wrong DAG. The latter is a corrupted version of the true DAG, where edges are altered. The Mean Absolute Residual is calculated by averaging the X and Y absolute residuals per sample. The LHS panel shows the results for an under-parametrized model, while the RHS for an over-parametrized one. Over-parametrization enables the transformer to perfectly learn the noise, even with the wrong DAG, and makes it hard to notice dependence in the scatter plot. [Experiments `54694937`, `54694944`, `stage_SoftMax_hard_wrong_scm6`, `54603903`, Dataset `scm6`.]

Regime	Experiment	S variable	HSIC (mean ± std)E-5
over-par.	true DAG	S_1	(5.25 ± 1.81)
		S_2	(4.32 ± 1.89)
		S_3	(4.41 ± 1.62)
	wrong DAG	S_1	(104.48 ± 107.56)
		S_2	(51.25 ± 71.13)
		S_3	(52.56 ± 48.89)
under-par.	true DAG	S_1	(943.64 ± 2072.06) †
		S_2	(12.56 ± 15.39)
		S_3	(14.64 ± 11.10)
	wrong DAG	S_1	(895.90 ± 577.42)
		S_2	(188.82 ± 109.12)
		S_3	(102.75 ± 117.62)

Table 1: The table reports the HSIC averaged across cross-validation folds (k=5) and grouped by parametrization regime, provided DAG and source variable. HSIC highlights higher independence for "true DAG" in both regimes. The high mean and mostly standard deviation of † is mostly due to one out of five cross-validation folds resulting in a poor fit (test MSE 25X other folds).

5 Why Learning the DAG is Hard

Since the transformer is guaranteed to be intervention-invariant only when the true DAG is provided, and in practical applications, this is not always the case, we dedicate the final section of the report to analyzing the challenges in causal discovery with transformers.

Attention

The most popular activation function used in attention blocks has a major problem when learning a causal structure. As SoftMax enforces a unitary sum of attention scores for each query on its possible keys, it becomes impossible to learn the absence of an edge. In practice, every query will have at least one incoming edge from a parent key, resulting in a fully-connected DAG. In our two-decoder transformer, this problem mainly affects self-attention, as cross-attention encodes known (or reasonably assumed) causal hierarchies in which keys are always parents of some queries.

For the reasons above, we propose *Lie Attention*, an attention function based on the Lie commutator. Given queries, keys and scale Q, K and τ, the Lie attention is defined as

$$A = \mathrm{GeLU}\left(\frac{QK^\top - (QK^\top)^\top}{\tau}\right)V \tag{3}$$

and has the following desired properties.

- scores on the diagonal are zero by construction; tokens can only update their representations using other tokens, not themselves, eliminating possible cycles.
- tokens ij learn a directional relation whose sign has a causal meaning. Consider the score matrix $\alpha = QK^T - (QK^T)^\top$ in equation 3, $\alpha_{ij} = -\alpha_{ji}$ by construction. Given the edge $j \to i$, $\alpha_{ij} > 0$ and $\alpha_{ij} < 0$.
- zero attention is allowed, since we are not forcing any unitary sum of contributions. Therefore, it becomes possible for a source token not to update its representation based on the keys in the sequence, while maintaining its causal role as a parent.
- The GeLU keeps only the positive part, so that only children queries are updated from parent keys.

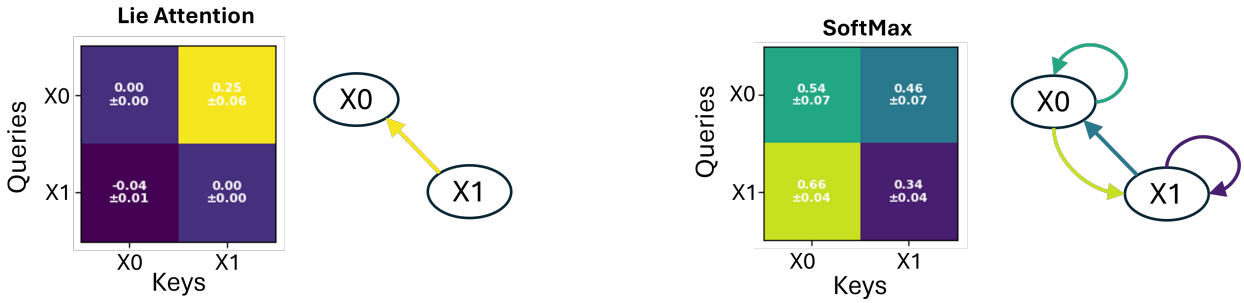


Figure 5: The figure illustrates the differences between the scores produced by the Lie Attention (LHS) and by the SoftMax (RHS), with their learned causal relations.

As Lie Attention requires square matrices, it can only be applied in self-attention blocks. For cross attention, we can relax the unitary sum by simply replacing the SoftMax by a another non-linear function. We then use the *Causal Cross Attention* as

$$A = \mathrm{GeLU}\left(\tanh\left(QK^\top/\tau\right)\right)V \tag{4}$$

with the following properties.

- Zero dot product between queries and keys corresponds to zero attention, as in Lie Attention
- The GeLU removes most of the negative part, leaving a continuous transition region where gradients can still flow for a limited negative domain.
- The maximum value is one, and more than one key can score unitary attention for a given query. This naturally implies that a given child token may have multiple parents with maximum probability.

Synchronous Update

Generally, there is no order to parameter updates during optimization, and both source and sink representations move in parameter space simultaneously. In supervised learning, parameters are updated to explain correlations, and it is irrelevant whether the information flows from sources to sinks or vice versa.

We propose orthogonal source embeddings, such that $\langle \mathrm{emb}(S_i), \mathrm{emb}(S_j) \rangle = 0$ for $i \neq j$. This allows us to best disentangle source representation in the dot-product space of any dot-product-based attention score. The source embeddings are kept frozen during training to enforce the information flow from sources to sinks. In the case of multiple stages, a training scheduler should train and freeze a stage first, before moving to the next. This would ensure that the information flows downstream when subsequent stages encode structural causality.

Equivalent solutions in attention factorization

Attention is extremely flexible and can learn different representations that are compatible with the optimization objective. Figure 6 shows an example of the same training in two different cross-validation k-folds. For $k = 0$, the model learns the correlation between X_1 and X_2 by forcing X_1 to attend S_3 and X_2 to attend X_1. In the same training, but for $k = 1$, X_1 learned the dependence of X_2 by learning its parents S_2 and S_3.

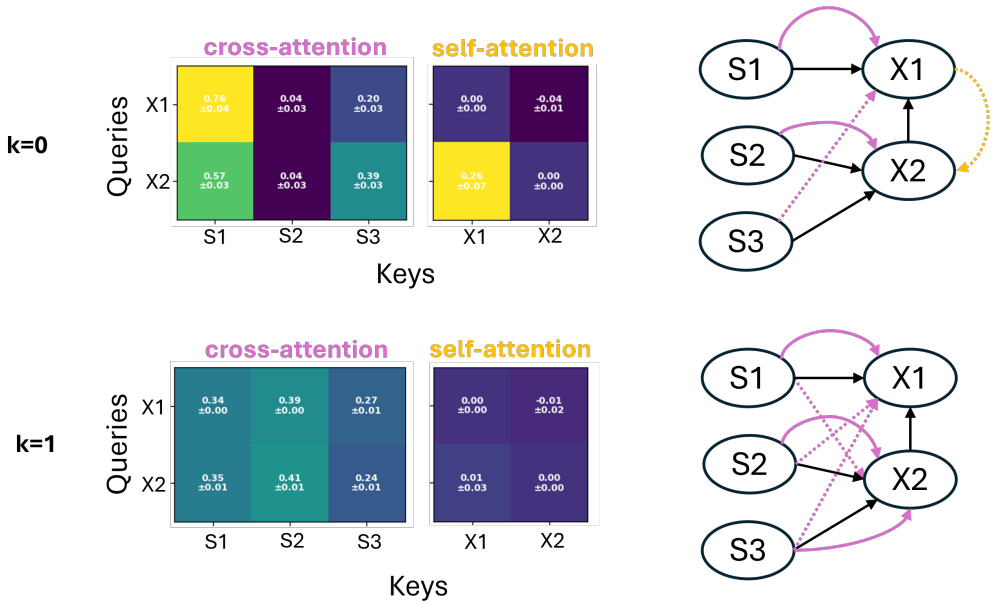


Figure 6: The figure shows two learned DAGs from the same model, in different cross-validation folds. Black edges are the ground truth, colored edges are the learned ones, purple for cross-attention and orange for self-attention. Correctly learned edges are solid lines, wrong ones are dotted.

This is currently an open problem for which we do not have a solution yet. One possible direction is to separate the embedding updates coming from self and cross attention, so that no representation leakages of X_2 are possible from S_2 and S_3.

***CausaliT*: a transformer for causal discovery and intervention invariance**

Combining the strategies discussed above, we implemented a transformer tailored for causal discovery. It is based on the inverted stage decoder of Figure 2, with Lie Attention and Causal Cross Attention. For simplicity, we focus on only the first stage $\mathbf{S} \rightharpoonup \mathbf{X}$.

In the attention block, scores $\alpha \in \mathbb{R}^{B \times L_Q \times L_K}$ are calculated for each sample in the batch used to update the running statistics $\alpha_{\text{mean}}, \alpha_{\text{std}} \in \mathbb{R}^{L_Q \times L_K}$. The empirical mean is used to calculate the prior $p_0 = \text{Sigmoid}(\alpha_{\text{mean}})$.

The DAG is explicitly learned as follows.

1. The DAG is initialized with α_{mean} of the first batch and normalized with a Sigmoid, obtaining probabilities p_ϕ.
2. The entries of the DAG represent the probability of an edge to be active; therefore, we sample, for each batch, a DAG whose entries $\sim \text{Bern}(p_\phi)$
3. The sampled DAG is applied to the attention output: only active edges allow information flow

We then formulate the problem as Bayesian optimization with prior p_0. The learned DAG probabilities p_ϕ are bounded to the evidence p_0 through the KL divergence. Further, we regularize the cost function using HSIC between the residuals and the inputs. Because lower HSIC corresponds to the true DAG, this term penalizes solutions that are far from it.

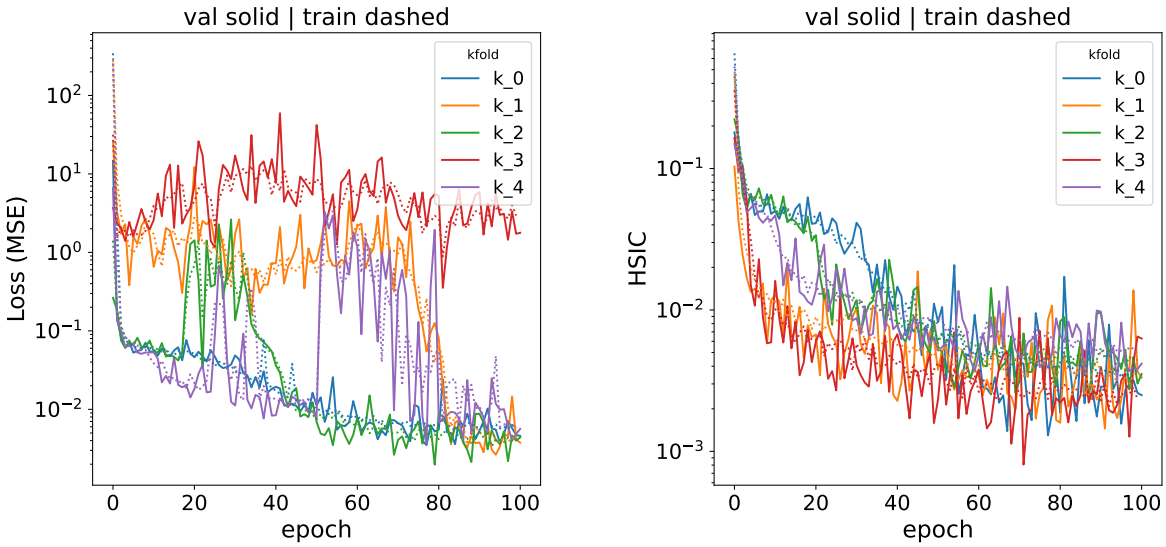
Altogether, the optimization problem has the following form.

$$\theta = \arg\min \Big(\underbrace{\mathrm{MSE}(\hat{\mathbf{X}}, \mathbf{X})}_{\text{fit data}} + \underbrace{\lambda_1 \, \mathrm{KL}(p_0 \| p_\phi)}_{\text{prior}} + \underbrace{\lambda_2 \, \mathrm{HSIC}(\mathbf{X}\text{-}\hat{\mathbf{X}}, \mathbf{S})}_{\perp \text{ residuals}} \Big) \tag{5}$$

NB: as we are considering only the first stage of the SCM, the input is $\mathbf{S}$ *and the target is* $\mathbf{X}$.

Measure intervention invariance

To conclude, the intervention-invariance of CausaliT is compared with that of the standard SoftMax to evaluate whether efforts to learn the DAG have a positive effect on the desired application. Since we focus only on the first stage for simplicity, we study $\mathbb{E}^{\mathrm{do}(S_1=s')}[X_2]$, with $s' \in \{-1, 0, 1\}$. Since $S_1 \notin \mathrm{PA}(X_2)$, the ground truth is $\mathbb{E}^{\mathrm{do}(S_1=s')}[X_2] = \mathbb{E}[X_2]$, i.e, zero deviations $\Delta X_2^{\mathrm{do}(S_1=s')}$. We trained with cross-validation ($k = 5$) a model with SoftMax attention and causaliT on the first stage $\mathbf{S} \rightharpoonup \mathbf{X}$ of the `scm6` dataset, performed predictions on the test split with and without interventions, and calculated the deviations as their difference. The results are presented in Figure 8.



(a) (b)

Figure 7: CausaliT training metrics. [Experiment `loc001`. Dataset `scm6`.]

Results

- Cross-validation data splits lead to very different results in both models, with a slightly more pronounced difference in *CausaliT*. Comparing all cross-validation folds, *CausaliT* performs strictly worse than SoftMax but, by selecting the best fold from each model, it performs better.
- Training metrics for our model reveal substantial differences between cross-validation folds. As visible in Figure 7, the best fold $k = 3$ for intervention invariance features the worst loss and the best HSIC. This suggests the presence of mechanisms in the training that work against learning the true causal structure.

6 Conclusion & Outlook

To properly employ transformers as surrogate models for optimization, they need to be intervention-invariant. We observed that, when the true DAG is provided, the transformer naturally inherits this property. When no DAG is provided, or a wrong one is provided, the transformer fails, and interventions on non-ancestors can incorrectly affect a child's distribution. Motivated by the need for the true causal structure, we proposed CausaliT, a transformer tailored for causal discovery. The performance of our model is strongly influenced by training conditions, as observed in different cross-validation splits. The models with the lowest HSIC perform best on intervention invariance but also have the worst loss. The linear-Gaussian SCM, despite its simplicity, poses a significant challenge for the transformer. In a more realistic non-linear scenario, we expect better system identifiability and, therefore, better performance of our causal-friendly architecture (see teaser in A).

Finally, given the current status of the project[3], we wish to intrigue the reader with the following final question that outlines our direction, warmly inviting them to follow our future developments.

> ***"Which training conditions help learning the true causal structure, and which metrics are the most informative to select the best cross-validation fold?"***

[3] Project repo https://github.com/scipi1/causaliT

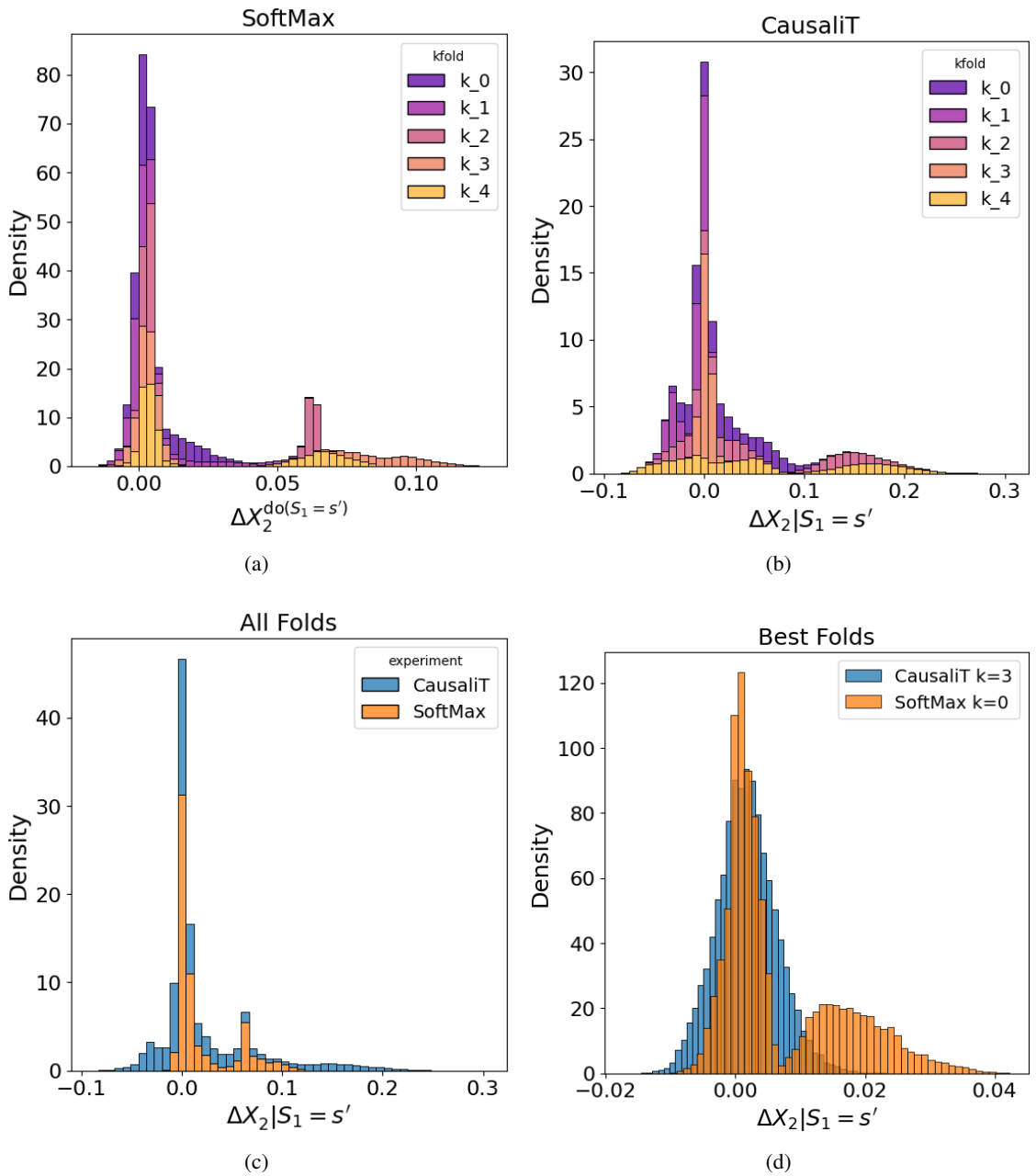


Figure 8: Intervention experiments for CausaliT and SoftMax. The top panels show the deviations of X_2 under intervention in different cross-validation folds for the SoftMax (LHS) and CausaliT (RHS). The bottom panels directly compare the two models in all cross-validation folds (LHS) and the best fold (RHS). The best fold has been selected as the one with the least deviation from zero. In all plots, the ground truth is zero, as $\Delta X_2^{\mathrm{do}(S_1:=s)} = 0,\ \forall s$. [Experiments `loc001`, `54946380`. Dataset `scm6`.]

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023.

[2] Manqing Liu and David R Bellamy. DAG-aware Transformer for Causal Effect Estimation.

[3] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting, March 2021.

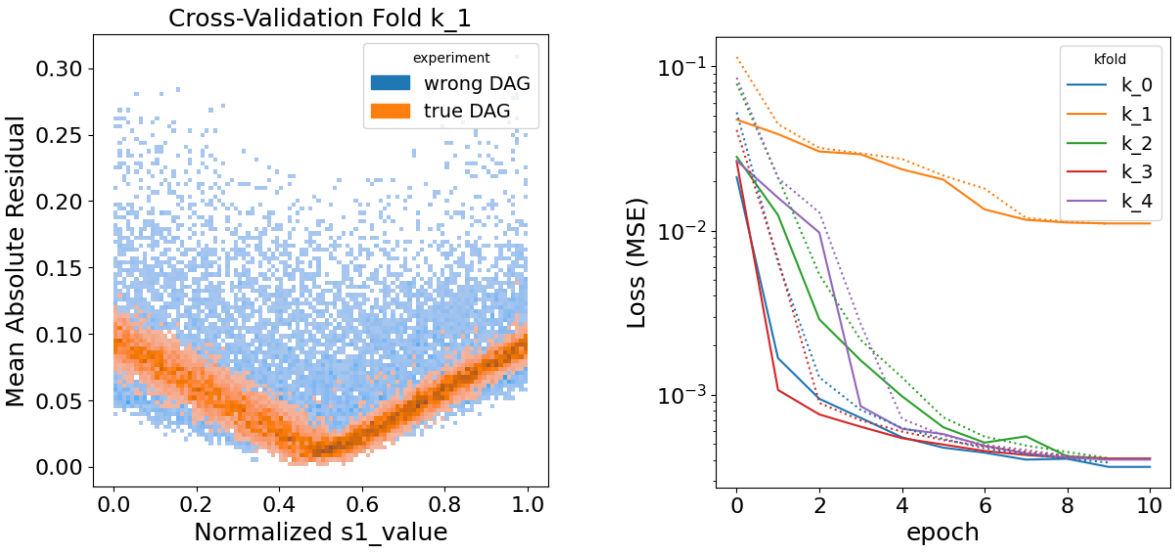
[4] Yao-Hung Hubert Tsai, Shaojie Bai, Makoto Yamada, Louis-Philippe Morency, and Ruslan Salakhutdinov. Transformer Dissection: A Unified Understanding of Transformer's Attention via the Lens of Kernel, November 2019.

[5] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural Machine Translation by Jointly Learning to Align and Translate, May 2016.

[6] Ramalingam Shanmugam. Elements of causal inference: Foundations and learning algorithms. *Journal of Statistical Computation and Simulation*, 88(16):3248–3248, November 2018.

A Complementary Material

Dependence triggered by poor fit



(a) Residuals

(b) Validation solid, train dotted

Figure 9: Under-parametrized experiment `54694944`, dataset `scm6`, focus on k_1 cross-validation fold, resulting in poor training and high residual-S_1 dependence.

Teaser: non-linear SCM

We repeated the intervention experiment with a non-linear SCM. In this scenario, CausaliT provides a better intervention invariance than the SoftMax, compared to the linear case.

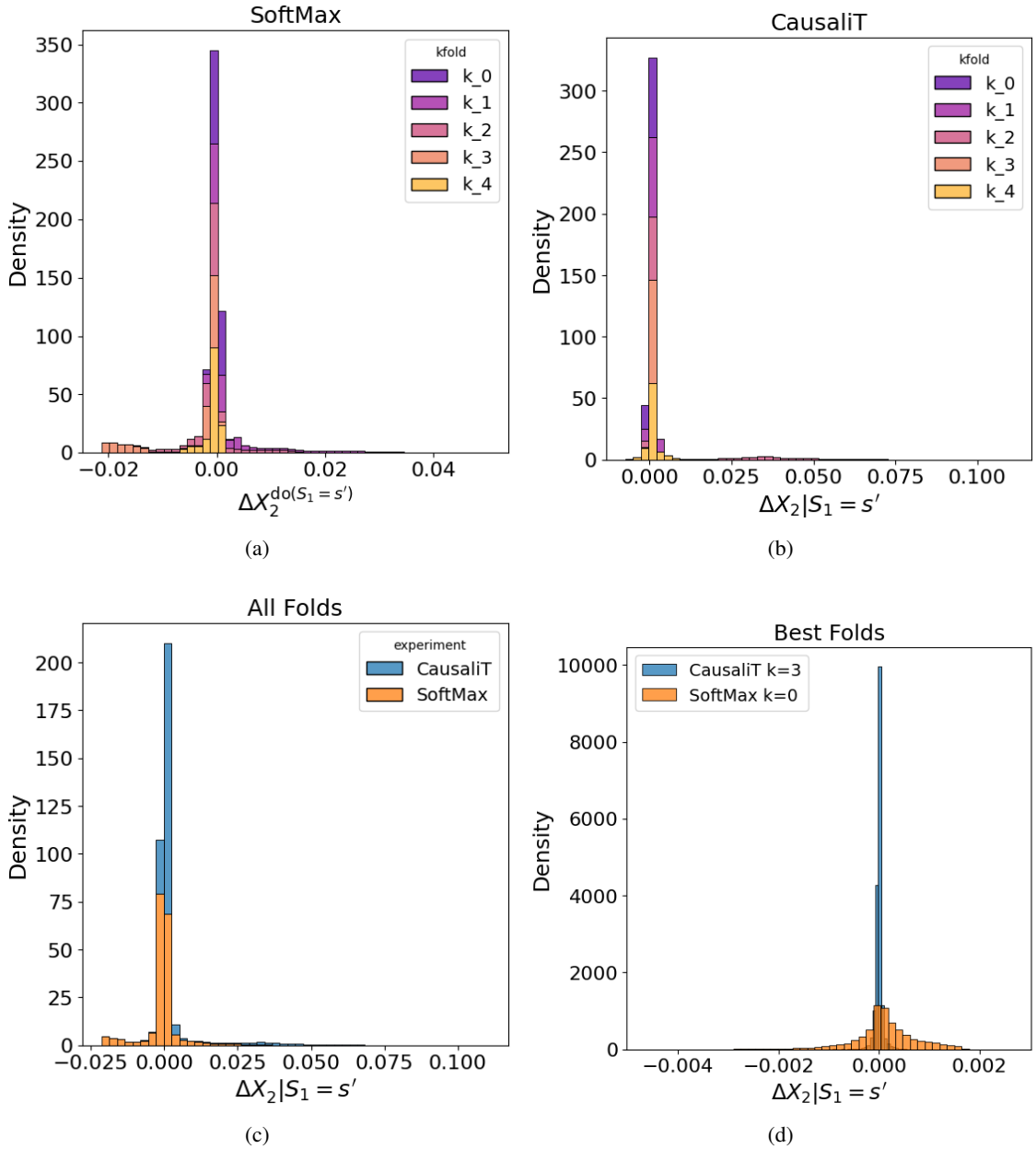


Figure 10: Intervention experiments for CausaliT and SoftMax. The top panels show the deviations of X_2 under intervention in different cross-validation folds for the SoftMax (LHS) and CausaliT (RHS). The bottom panels directly compare the two models in all cross-validation folds (LHS) and the best fold (RHS). The best fold has been selected as the one with the least deviation from zero. In these experiments, a non-linear SCM dataset was used. In all plots, the ground truth is zero, as $\Delta X_2^{\mathrm{do}(S_1:=s)} = 0,\ \forall s$. [Experiments 55058272, 55058353. Dataset `scm7`.]